

PROBE-AUFGABENSTELLUNG

Dieses Dokument dient zur Erläuterung des Assignment-Prinzips in der Vorlesung. Die finale Version enthält den gruppenspezifischen Datensatz.

Assignment: Systematischer Modellvergleich

mit klassischen Modellen und neuronalen Netzen

Kurs: Maschinelles Lernen mit Python, scikit-learn und Keras

Hochschule: DHBW Stuttgart

Semester: Sommersemester 2026

Bearbeitungszeit: TBD

Gruppengröße: max. 4 Personen

Abgabe: TBD

1. Aufgabenüberblick

Jede Gruppe erhält einen realen Datensatz aus der Praxis. Ihre Aufgabe ist es, eine vollständige Machine-Learning-Pipeline durchzuführen und dabei fünf Modelle systematisch zu vergleichen: drei klassische Modelle aus dem Kurs sowie zwei Varianten eines neuronalen Netzes (MLP).

Ziel ist nicht das bestmögliche Ergebnis, sondern ein methodisch sauberes, nachvollziehbares und kritisch reflektiertes Vorgehen. Alle Entscheidungen müssen begründet werden.

2. Ihr Datensatz

Datensatz	Telco Customer Churn
Quelle	Kaggle: kaggle.com/datasets/blastchar/telco-customer-churn
Umfang	ca. 7.000 Samples 20 Features (numerisch und kategorisch)
Aufgabe	Binäre Klassifikation: Kündigt ein Kunde seinen Mobilfunkvertrag? (Yes / No)
Besonderheit	Kategorische Features müssen kodiert werden. Leichte Klassenimbalance (~27 % Churn). Diese Besonderheiten sind Teil der Aufgabe.

3. Zu vergleichende Modelle

Jede Gruppe vergleicht exakt die folgenden fünf Modelle in der angegebenen Reihenfolge:

#	Modell	Typ	Bezug zum Kurs
1	Logistic Regression	Klassisch	Kapitel 4
2	Decision Tree, Random Forest, XGBClassifier	Klassisch	Kapitel 5, wähle ein geeignetes Modell aus
3	k-NächsterNachbar oder Support Vector Machine	Klassisch	Kapitel 7, 8, wähle ein geeignetes Modell aus
4	Neuronales Netz Variante A	Neuronal	Kapitel 9
5	Neuronales Netz Variante B	Neuronal	Kapitel 9

Variante A:

Input → Dense(64, relu) → Dense(1, sigmoid)

Variante B:

Input → Dense(64, relu) → Dropout(0.3) → Dense(32, relu) → Dense(1, sigmoid)

4. Teilaufgaben

Bearbeiten Sie alle sieben Schritte vollständig und in der angegebenen Reihenfolge. Dokumentieren Sie jede Entscheidung direkt im Notebook mit Markdown-Zellen.

Schritt 1 Datenanalyse

- Datensatz laden, Form, Datentypen und fehlende Werte überblicken
- Verteilung der Zielvariable dokumentieren (absolut und prozentual)
- Mindestens 3 aussagekräftige Visualisierungen erstellen:
 - Verteilung numerischer Features (Histogramme)
 - Kategorische Features (Balkendiagramme)
 - Korrelationsmatrix
 - Zusammenhang zwischen Features und Zielvariable
- Welche Features erscheinen besonders relevant? Begründen Sie Ihre Einschätzung.

Schritt 2 Preprocessing und Datenvorbereitung

- Fehlende Werte identifizieren, behandeln und Entscheidung begründen
- Kategorische Features kodieren (One-Hot oder Label Encoding – mit Begründung)
- Stratifizierten Train / Validation / Test-Split durchführen (70 / 15 / 15)
- Features skalieren – welche Features und warum?
- Darf der Scaler auf dem gesamten Datensatz gefittet werden? Begründen Sie Ihre Antwort.

Schritt 3 Klassische Modelle (Logistic Regression, Decision Tree, SVM, o.ä.)

- Alle drei Modelle mit Default-Parametern auf den Trainingsdaten trainieren
- Evaluation mit 5-facher stratifizierter Kreuzvalidierung
- Für jedes Modell: Accuracy, F1-Score (weighted) und ROC-AUC berechnen
- Ergebnisse in die Vergleichstabelle (Schritt 7) eintragen
- Kurzen Kommentar zu jedem Modell im Notebook verfassen

Schritt 4 Variante A – Einfaches neuronales Netz

- Architektur: Input → Dense(64, relu) → Dense(1, sigmoid)
- Optimizer: Adam | Loss: binary_crossentropy | max. 100 Epochen | batch_size: 32
- Early Stopping: monitor='val_loss', patience=5, restore_best_weights=True

- Überprüfe die Modellarchitektur auf Eignung: Falls ungeeignet, wähle eine optimierte Variante.
- Lernkurven (Loss und Accuracy) plotten und kommentieren:
 - Gibt es Hinweise auf Overfitting oder Underfitting?
 - In welcher Epoche hat Early Stopping abgebrochen?
- Accuracy und F1 auf dem Validierungsset dokumentieren

Schritt 5 Variante B – Tieferes Netz mit Regularisierung

- Architektur: Input → Dense(64, relu) → Dropout(0.3) → Dense(32, relu) → Dense(1, sigmoid)
- Optimizer: SGD (statt Adam) | Loss: binary_crossentropy | max. 100 Epochen | batch_size: 32
- Early Stopping identisch zu Variante A
- Überprüfe die Modellarchitektur auf Eignung: Falls ungeeignet, wähle eine optimierte Variante.
- Lernkurven plotten und mit Variante A vergleichen:
 - Was verändert der Dropout in den Lernkurven?
 - Warum konvergiert SGD anders als Adam?
- Zentrale Frage: Warum ist Variante B nicht automatisch besser als Variante A?

Schritt 6 Modellvergleich, Optimierung und finale Evaluation

- Vergleichstabelle aller Modelle auf dem Validierungsset ausfüllen:

Modell	Typ	Accuracy	F1	Notizen
Logistic Regression	Klassisch	—	—	
Decision Tree	Klassisch	—	—	
SVM	Klassisch	—	—	
MLP Variante A	Neural Net	—	—	
MLP Variante B	Neural Net	—	—	

- Hyperparameter-Optimierung für das beste Modell:
 - Klassische Modelle: GridSearchCV (mind. 2 Parameter)
 - Neuronale Netze: KerasTuner oder manuelle Suche über Neuronenzahl und Lernrate
- Finale Evaluation des optimierten Modells exklusiv auf dem Testset – nur einmal!
- Konfusionsmatrix des besten Modells ausgeben und interpretieren:
 - Wie viele False Positives und False Negatives gibt es?
 - Was bedeutet das im Praxiskontext Ihres Datensatzes?

Schritt 7 Reflexion und Fazit

- Welches Modell empfehlen Sie für den Produktiveinsatz – und warum?
- Was hat Sie im Verlauf der Arbeit überrascht?

- Wo sehen Sie Grenzen des besten Modells im Praxiskontext?
- Was würden Sie mit mehr Zeit, mehr Daten oder mehr Ressourcen anders machen?
- Bonusfrage (optional): Hätte ein CNN oder Transfer Learning hier geholfen – warum (nicht)?

5. Abgabe und Deliverables

Bitte reichen Sie folgende Dokumente bis zum [DATUM EINTRAGEN] ein:

1	Google Colab Notebook	.ipynb – vollständig ausgeführt, alle Zellen mit Markdown-Kommentaren versehen
2	PDF-Export des Notebooks	Direkt aus Colab: Datei → Drucken → Als PDF speichern
3	Management Summary	Zusammenfassung der wesentlichen Ergebnisse

6. Hinweise zur KI-Nutzung

KI-Tools sind ausdrücklich erlaubt.

ChatGPT, Claude, GitHub Copilot und ähnliche Werkzeuge dürfen für Code, Debugging und Erklärungen eingesetzt werden. Es gelten folgende Regeln:

- Jeden KI-generierten Codeblock im Notebook kennzeichnen: # Quelle: [Tool-Name], z. B. # Quelle: ChatGPT
- KI-generierter Code muss vollständig verstanden werden. Bei Rückfragen muss jeder Code-Abschnitt erklärt werden können.
- Interpretationen, Entscheidungsbegründungen und Reflexionen müssen eigenständig verfasst werden. KI darf als Diskussionspartner dienen, nicht als Autor.
- Die finale Empfehlung im letzten Schritt trifft die Gruppe – nicht die KI.
- Wer nicht erklären kann, was sein Code tut, hat ihn nicht verstanden. Das ist kein KI-Problem, sondern ein Lernproblem.

7. Praktische Tipps

- Beginnen Sie mit der Datenanalyse – sie gibt Ihnen das nötige Verständnis für alle folgenden Entscheidungen.
- Kommentieren Sie jede Designentscheidung direkt im Notebook mit einer Markdown-Zelle. Später erinnern Sie sich nicht mehr, warum Sie etwas getan haben.
- Nutzen Sie Google Colab mit GPU-Laufzeit (Laufzeit → Laufzeittyp ändern → T4 GPU) für die Keras-Modelle.
- Schauen Sie sich die Konfusionsmatrix genau an – sie sagt oft mehr als die Accuracy allein.
- Faustregel: Das Testset wird exakt einmal angefasst – nach allen Entscheidungen.

PROBE-DOKUMENT | Datensatz und Abgabedaten werden in der finalen Version angepasst
DHBW Stuttgart | Maschinelles Lernen mit Python, scikit-learn und Keras | SS 2026